4、MoveIt运动学设计

4、MoveIt运动学设计

- 4.1、三维空间的位姿描述
- 4.2、启动
- 4.3、反解关键代码
  - 4.3.1、C++代码
- 4.4、正解关键代码
  - 4.4.1、C++代码
- 4.5、MoveIT运动学插件

本节课以MoveIT仿真为例，如果需要设置真机与仿真同步，请看【02、MoveIt注意事项与控制真机】这节课。！！！注意安全！！！

效果演示为虚拟机，其他主控运行情况（与主控性能有关，根据实际情况而定）。

4.1、三维空间的位姿描述

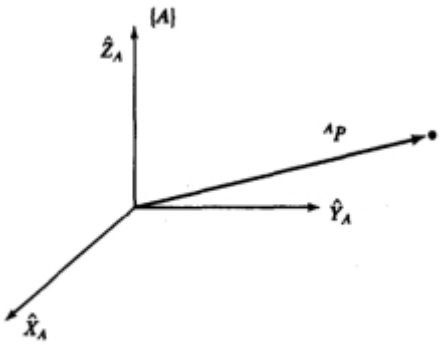
首先规定一个坐标系，相对于该坐标系，点的位置可以用3维列向量表示;刚体的方位可用3×3的旋转矩阵来表示。而4×4的齐次变换矩阵则可将刚体位置和姿态（位姿）的描述统一起来，它具有以下优点：

(1) 它可描述刚体的位姿，描述坐标系的相对位姿（描述）。

(2) 它可表示点从一个坐标系的描述转换到另一坐标系的描述（映射）。

(3)它可表示刚体运动前、后位姿描述的变换（算子）。

- 位置描述——位置矢量

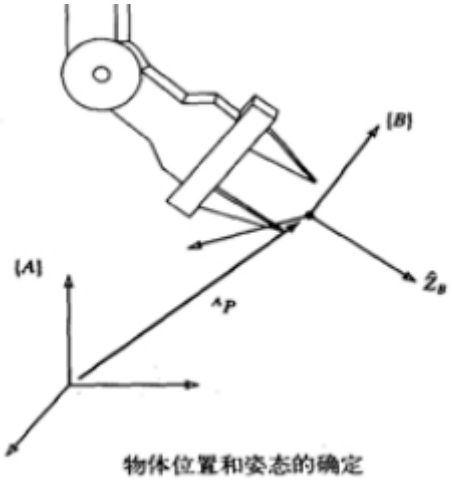


相对于坐标系的矢量（示例）

用三个相互正交的带有箭头的单位矢量来表示一个坐标系{A}。那么点p在坐标系{A}下空间位置的表示为：

$$^AP = \begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix}$$

- 方位的描述——旋转矩阵



物体位置和姿态的确定

常用的旋转变换矩阵是绕X轴、绕Y轴或绕Z轴转一角度。它们分别是

$$R(X,\theta)=\begin{bmatrix}1 & 0 & 0\\ 0 & cos(\theta) & -sin(\theta)\\ 0 & sin(\theta) & cos(\theta)\end{bmatrix} R(Y,\theta)=\begin{bmatrix}cos(\theta) & 0 & sin(\theta)\\ 0 & 1 & 0\\ -sin(\theta) & 0 & cos(\theta)\end{bmatrix} R(Z,\theta)=\begin{bmatrix}cos(\theta) & -sin(\theta) & 0\\ sin(\theta) & cos(\theta) & 0\\ 0 & 0 & 1\end{bmatrix}$$

- 非齐次表述

向量a经过一次旋转R和一次平移t后，得到$a' : a'=R*a+t$

- 齐次表述

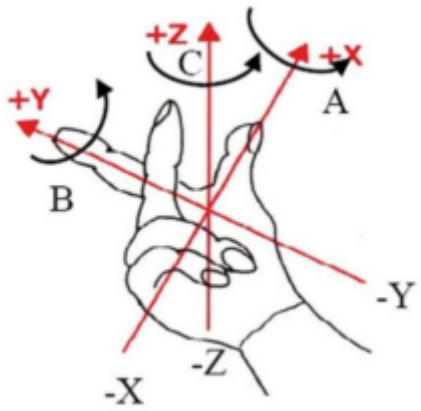
$$\begin{bmatrix}a\prime\\ 1\end{bmatrix}=\begin{bmatrix}R & t\\ 0^T & 1\end{bmatrix}\begin{bmatrix}a\\ 1\end{bmatrix}=T\begin{bmatrix}a\\ 1\end{bmatrix}$$

通用旋转矩阵R为：

$$R=R_z(\beta)R_y(\alpha)R_x(\theta)$$

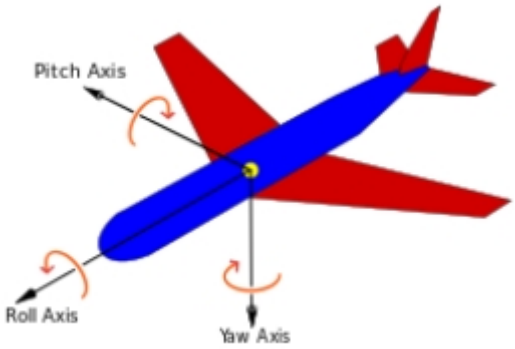
这里的旋转矩阵旋转过程，先绕X轴旋转θ角，然后绕Y轴旋转α角，最后绕Z轴旋转β角。

- 右手法则



大拇指指向X，食指伸向Y，中指的方向是Z。

- RPY与欧拉角



按照自身坐标系旋转顺序是 Z-->Y-->X，叫做eulerYPR（Yaw Pitch Roll）；

按照外界坐标系（参考坐标系）的旋转顺序是x-->y-->z，叫做RPY（Roll Pitch Yaw）；

描述同一姿态时，以上两者的表示是等价的，即Yaw Pitch Roll的角度值是一样的。

定义： Yaw 偏航角 ϒ ; Pitch 俯仰角 β ; Roll 翻滚角 α

- 四元数法

大概的定义是，用标量和三个虚轴表示向量，叫四元数。[x, y, z, w]每个数据的范围均是[-1,1]。

4.2、启动

正解 : 在一个机械臂中，已知每个关节角度，求解末端位姿。

反解 : 在一个机械臂中，已知末端(除夹爪)位姿，求每个关节角度。

启动MoveIT

```
roslaunch arm_moveit_demo x3plus_moveit_demo.launch sim:=true
```

启动反解节点

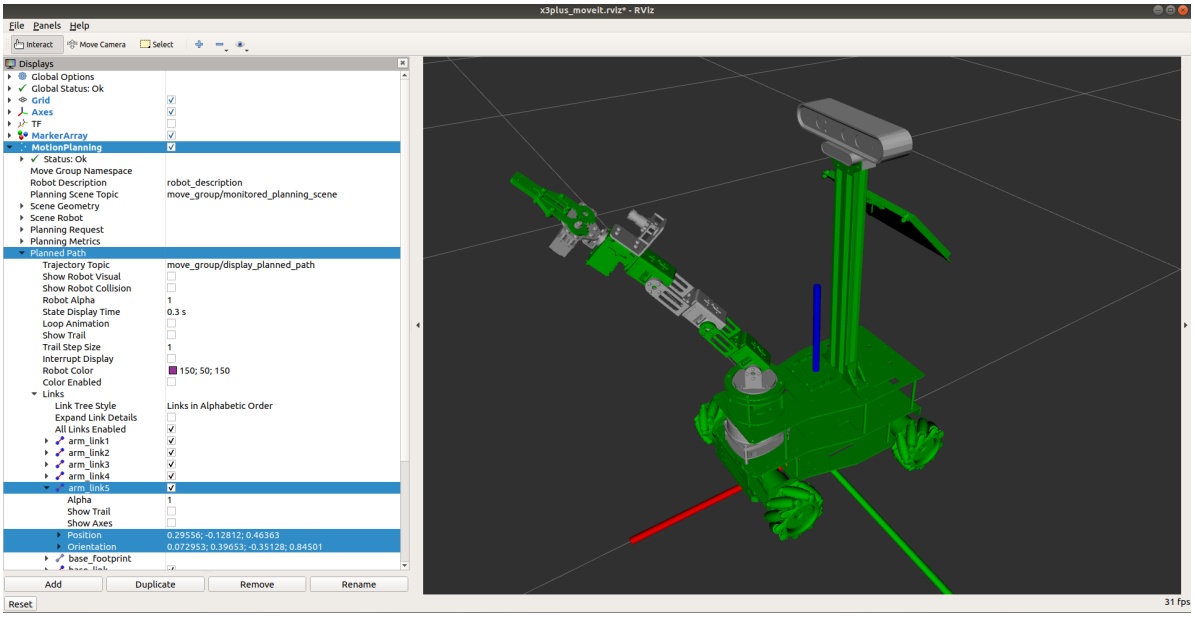
```
rosrun arm_moveit_demo 02_set_pose_plan
```

启动正解节点

```
rosrun arm_moveit_demo 03_set_joint_plan
```

4.3、反解关键代码

注意：由于机械臂的范围限制，有可能到达不了指定的目标位姿，循环多次执行只能略微提高规划成功率。不仅要考虑位置，还要考虑末端的姿态。我们可以通过rviz查看各个link的位姿，如下图，



上图中【arm_link5】中的【Position】、【Orientation】就是末端的位姿，使用反解的时候，也可以直接设置目标点的位置和姿态四元数。

4.3.1、C++代码

设置目标位姿

```
    //设置具体位置
    geometry_msgs::Pose pose;
    pose.position.x = 0.18721125717113798;
    pose.position.y = -0.008718652395814977;
    pose.position.z = 0.4787351295417709;
    // 设置目标姿态
    tf::Quaternion quaternion;
    // RPY的单位是角度值
    //double Roll = -180;
    //double Pitch = 45;
    //double Yaw = -180;
    // RPY转四元数
    //quaternion.setRPY(Roll * DE2RA, Pitch * DE2RA, Yaw * DE2RA);
    pose.orientation.x = 0.08637313729188083;
    pose.orientation.y = 0.40819551903689105;
    pose.orientation.z = 0.10050372779920543;
    pose.orientation.w = 0.9032248336328139;
    yahboomcar.setPoseTarget(pose);
```

循环执行路径规划

```
    int index = 0;
    // 多次执行,提高成功率
    while (index <= 10) {
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        // 运动规划
        const moveit::planning_interface::MoveItErrorCode &code =
yahboomcar.plan(plan);
        if (code == code.SUCCESS) {
            ROS_INFO_STREAM("plan success");
            yahboomcar.execute(plan);
```

```
            break;
        } else {
            ROS_INFO_STREAM("plan error");
        }
        index++;
    }
```

4.4、正解关键代码

4.4.1、C++代码

设置目标点

```
    //设置具体位置
    vector<double> pose{0, 0.79, -1.57, -1.57, 0};
    yahboomcar.setJointValueTarget(pose);
```

循环执行路径规划

```
    moveit::planning_interface::MoveGroupInterface::Plan plan;
    const moveit::planning_interface::MoveItErrorCode &code =
yahboomcar.plan(plan);
    if (code == code.SUCCESS) {
        ROS_INFO_STREAM("plan success");
        // 显示轨迹
        string frame = yahboomcar.getPlanningFrame();
        moveit_visual_tools::MoveItVisualTools tool(frame);
        tool.deleteAllMarkers();
        tool.publishTrajectoryLine(plan.trajectory_,
yahboomcar.getCurrentState()->getJointModelGroup("arm_group"));
        tool.trigger();
        yahboomcar.execute(plan);
    } else {
        ROS_INFO_STREAM("plan error");
    }
```

4.5、MoveIT运动学插件

MoveIT运动学插件是【kinematics.yaml】文件

如需更换或者修改，直接修改【kinematics.yaml】文件即可

```
roscd x3plus_moveit_config/config
sudo vim kinematics.yaml
```

- kdl

```
arm_group:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
```

- trac_ik

```
arm_group:
  kinematics_solver: trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
```